coding etiquette for (non-coder) Social Scientists

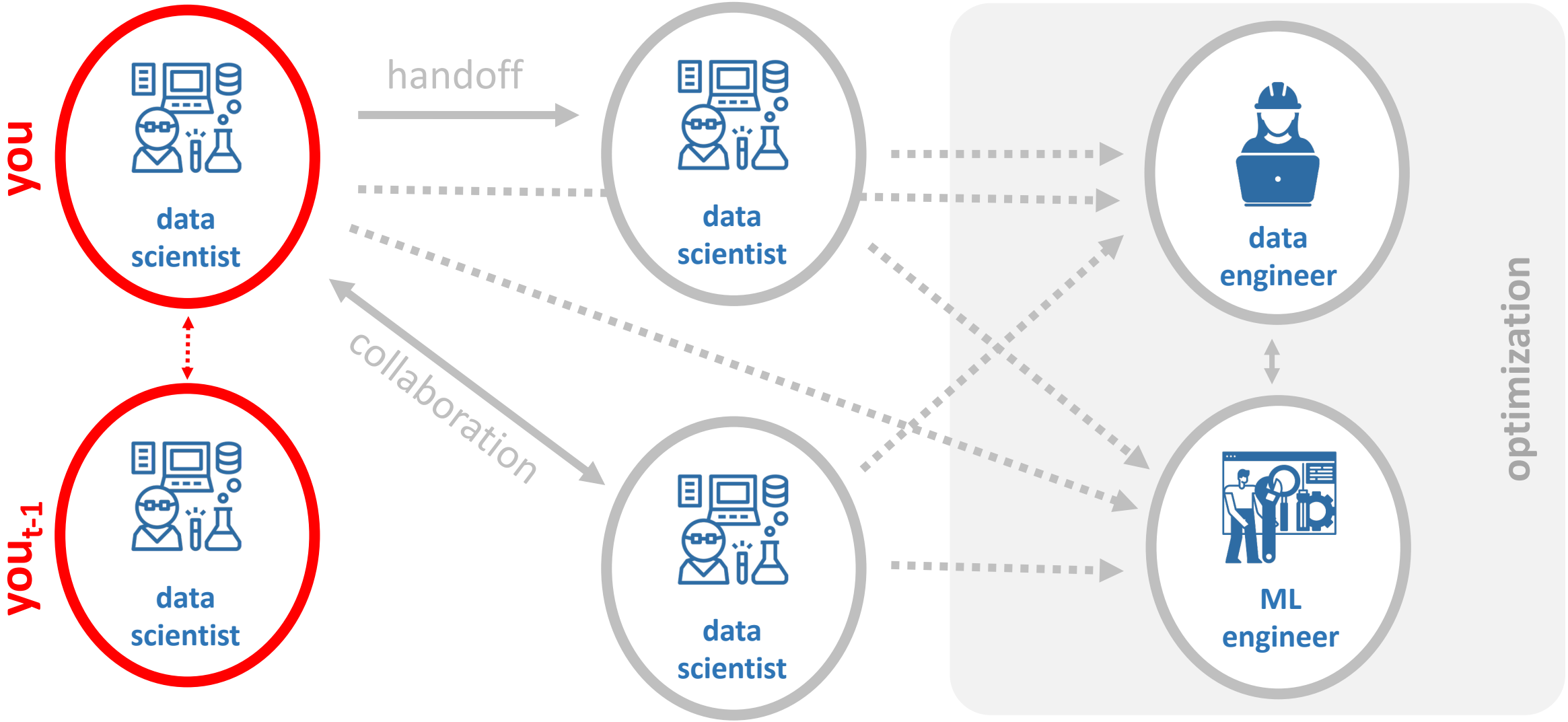
Marco Morales
marco.morales@columbia.edu

Nana Yaw Essuman
ne2388@columbia.edu

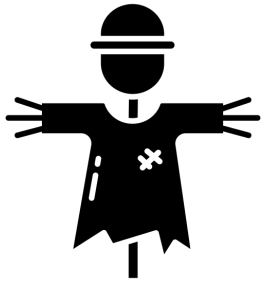
GR5069: Applied Data Science
for Social Scientists

Spring 2026
Columbia University

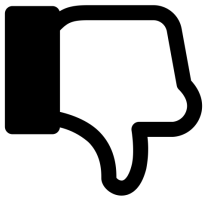
recap: workflow collaboration in Data Science



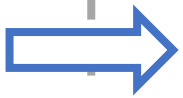
recap: iteration to build Data Products

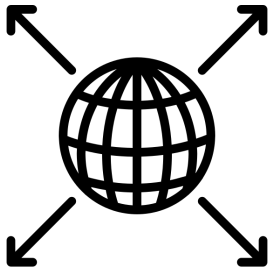


start small
(MVP)

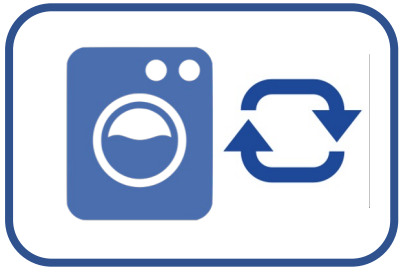


fail fast



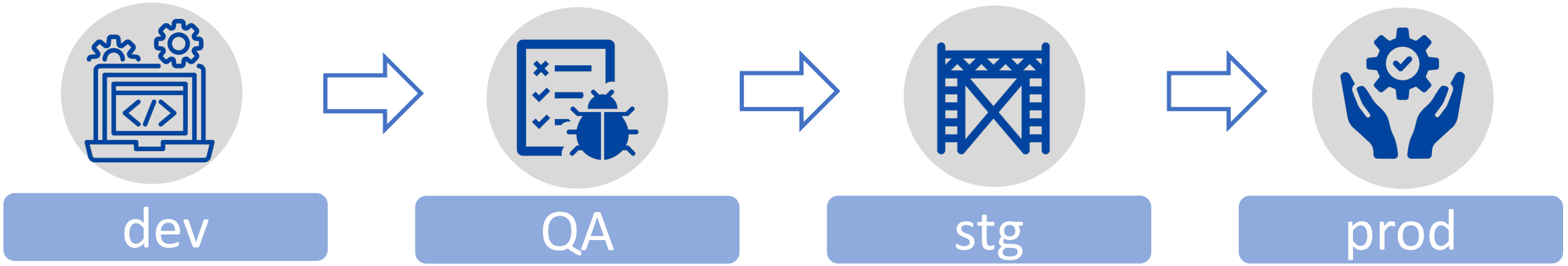


scale up



iterate

recap: working environments



development

quality assurance / testing

staging

production

recap: operational concepts in Data Science

portability

anyone should be able to **pick up where you left off** from any machine

replicability

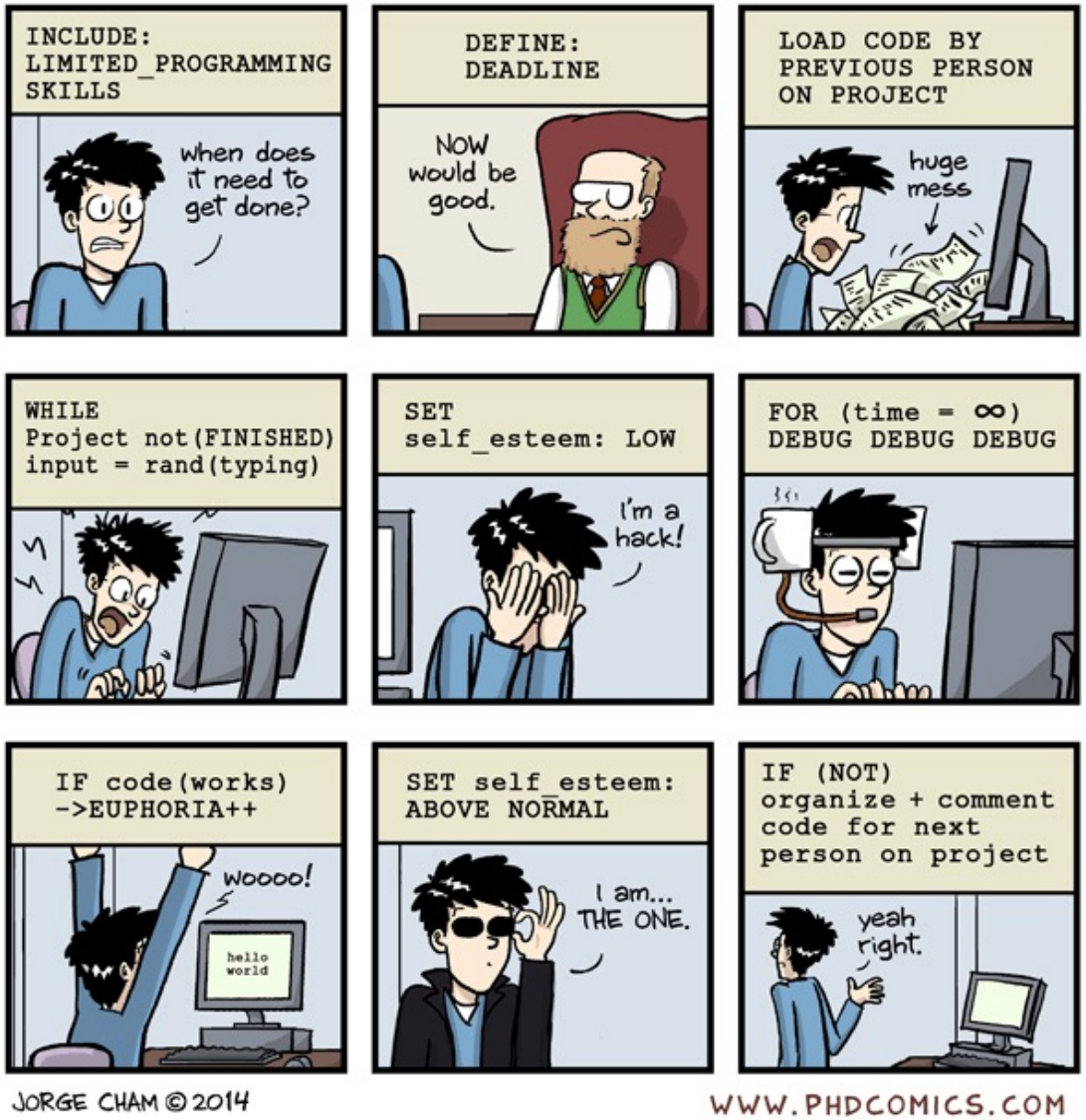
anyone should be able to arrive at your **same results**

scalability

your prototype should also work for **larger data sets** and/or be on the path of **automation**

our focus :

we've all done it, but… we shouldn't!



“good” code – the backbone of a “good” project

```
// Dear programmer:
// When I wrote this code, only god and
// I knew how it worked.
// Now, only god knows it!
//
// Therefore, if you are trying to optimize
// this routine and it fails (most surely),
// please increase this counter as a
// warning for the next person:
//
// total_hours_wasted_here = 254
//
```

what defines **“good” code** in Data Science?

1. easy to **read** (by you or any first-time user)
2. structured in **predefined sections** / **modules** / **scripts**
3. avoids **unnecessary complexity** (does not try to be clever)
4. does **exactly what it’s supposed to do** (nothing more, nothing less)
5. can handle **exceptions** and **edge cases** (without breaking)
6. easy to add **new features** (simple and complex ones)
7. easy to **maintain** (update to newer language/library versions)

why do we need **“good” code** in Data Science?

- VERY LIKELY, your code will end up **integrating** with **larger systems**
- also, **prototypes** tend to **scale** quite often
- best way to **be nice** to your teammates (and the you of the future)

remember:

- “good” code **does not need to be perfect**
  - but it **shouldn’t make portability**, **replicability** or **scalability harder**!
  - it just needs **baseline structure** (today’s topic!)

where things can (and will) go wrong!

- we write **“good” code** because we also want to **minimize problems down the line** when there are:
  - **data changes** – upstream schema changes, corrupted data
  - **library/package updates** – code may not produce the same output with new libraries or not work at all
  - **programming language updates** – syntax may need to be updated
  - **hardware updates** – may need additional changes to the codebase

“good” code should not be an afterthought!

- force yourself to **integrate “good” code principles** as part of your **everyday routine**
- the **most important** part of writing **“good” code** is **<u>consistency</u>**
  - within ***function*** > within ***module*** > within ***project***
- **don’t wait until the last minute** to follow “good” code principles
  - very likely you **won’t have time to go back** and redo your code

where to focus to write “good” code?

script structure

code syntax

code documentation

code validation

it all starts with your **scripts**…

good to know…

- (Jupyter/Markdown) **notebooks** are great for **sharing** work and (code) review
  - nice sandbox to **develop / test** code
  - nice way to **review code + outputs** without having to run it
  - (usually) **terrible for scaling**!

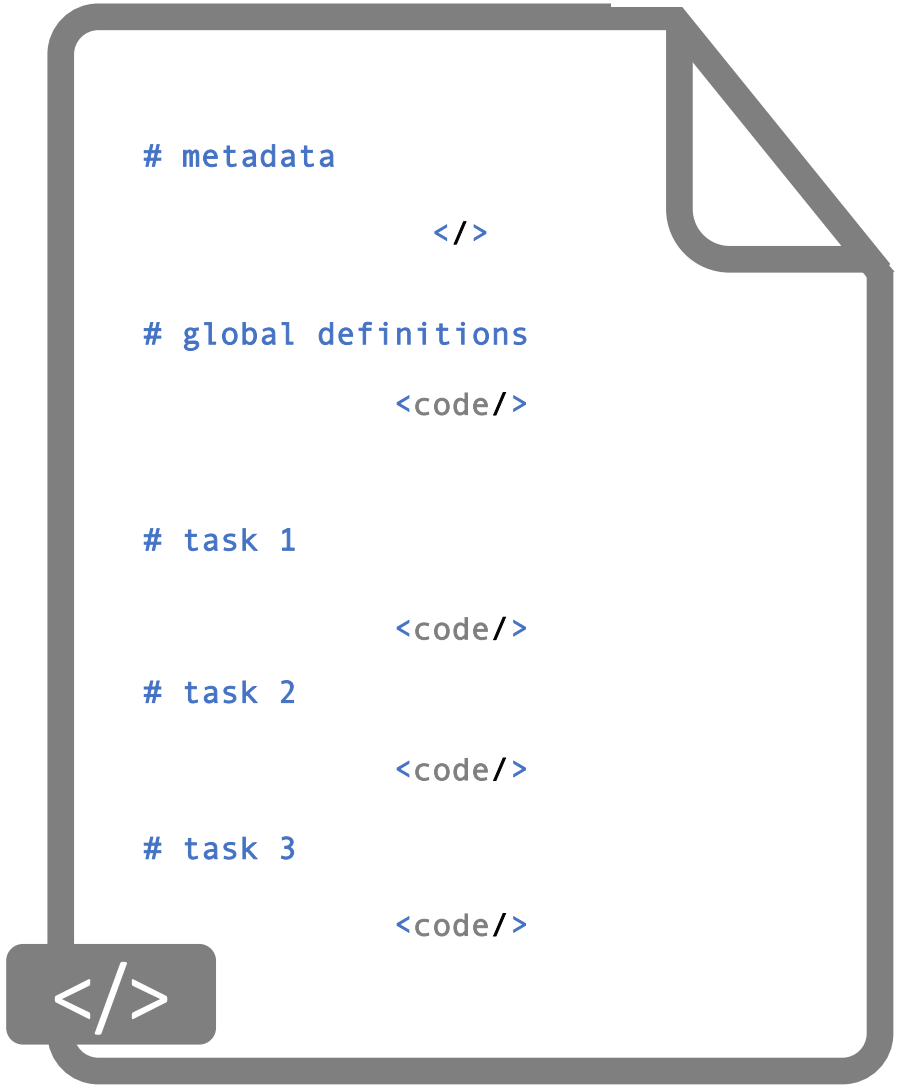
- **scripts** are preferred for **running processes** (especially in structured working environments)
  - scripts can be run directly from source

good to know…

- we'll call them **scripts** from here on
  - apply **same principles** if you started from a **notebook**

- creating a **structured notebook** makes it <u>easier</u> to extract a nearly finished **structured script**
  - you may need to extract your code **from a notebook into a script** if you developed it there

always create **structured scripts**



(1) start with a **meaningful metadata** section

```
# metadata
            </>

# global definitions
          <code/>

# task 1
          <code/>
# task 2
          <code/>
# task 3
          <code/>
```

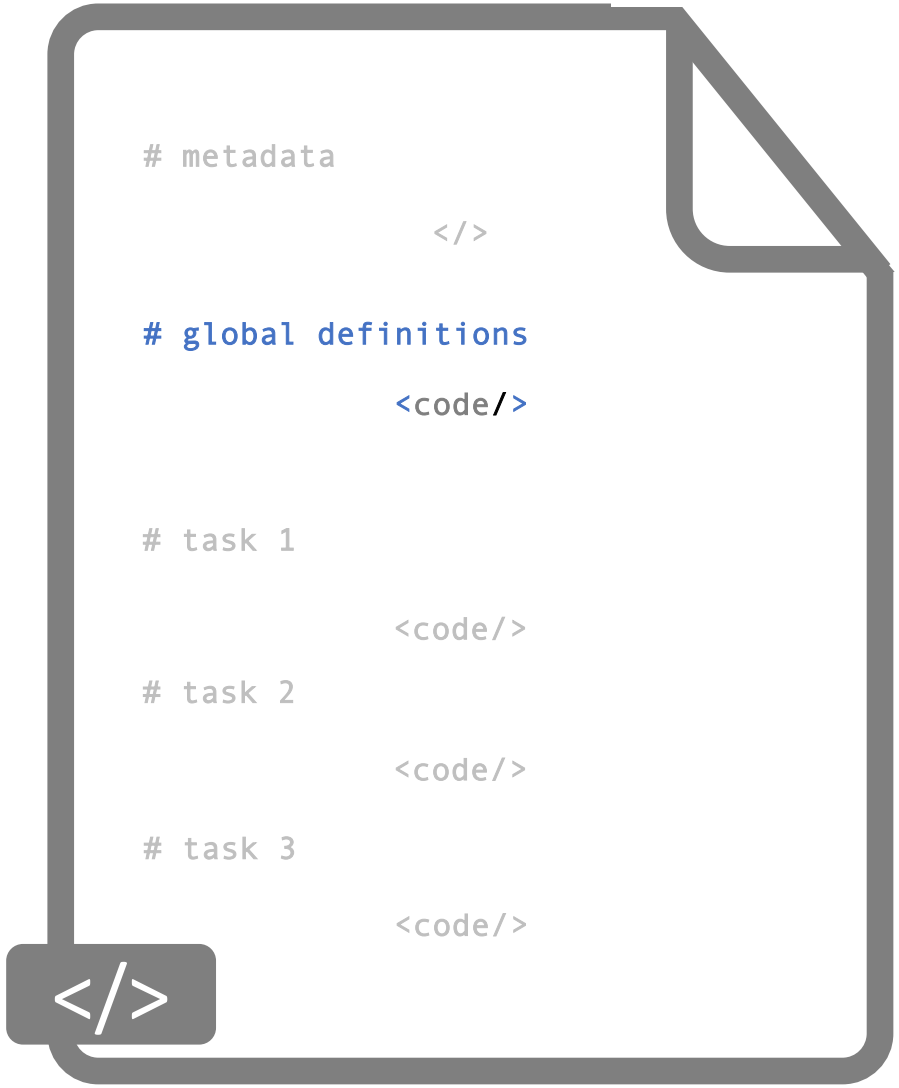
</>

(1) start with a **meaningful metadata** section

- things have changed wrt the meaning of “meaningful”
- most **metadata** for your script (author, email, change history) is baked in **git/GitHub**
- **minimal information** is now needed at the start of your script (language version, short purpose description)

```
#!/usr/bin/env python3
"""
User authentication module for the web application.
"""
```

(2) add a **global definitions** section



(2) add a **global definitions** section

- identify all **important definitions** that will be used in your script / project
- definitions include:
  - **libraries/packages to use**
  - **additional scripts**
  - **objects**
  - **parameters**
  - **functions**
- place them in a **single section** at the beginning of the script

(2) add a **global definitions** section

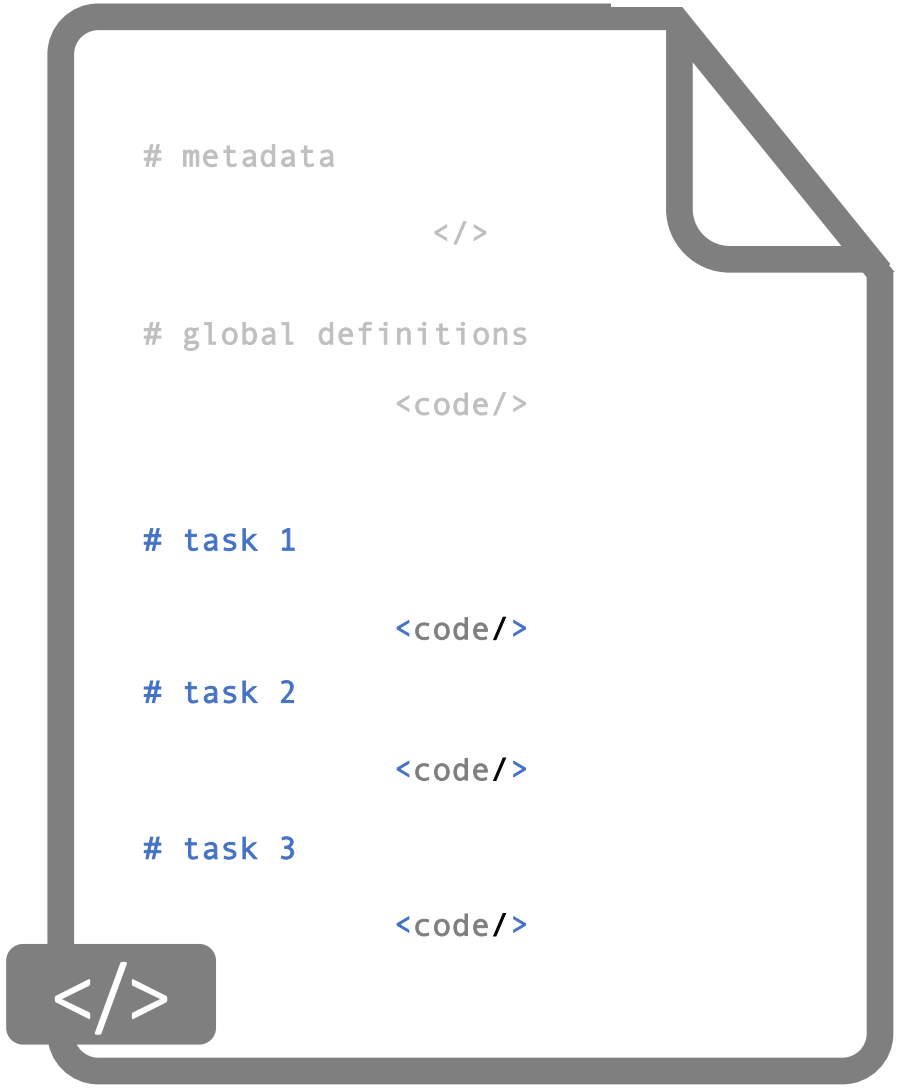
```python
# ======== GLOBAL DEFINITIONS ================================

# Packages to import
import pandas as pd
from pathlib import Path

# Additional scripts to import
from modeling_functions import *

# Data paths
raw_data_confrontations = Path("data") / "raw" / "A-E.xlsx"
equivalence_table = Path("data") / "external" / "ARCH535.csv"

# Model parameters
MAX_ITERATIONS = 1000 # number of iterations for convergence
CONVERGENCE_TOLERANCE = 1e-6 # convergence threshold
```

(2) add a **global definitions** section

ProTips:

- do **not** add **definitions at different places** in the code – collect them in the same place!
- always use **relative paths** to define **locations** and **files**
- place at **beginning** of the script if using a single **short script**
- place on a **separate** `config` file if working on **larger projects**

(3) break down **tasks** in **sections**



(3) break down tasks in sections

- **“title”** each section with a **brief description**
  - “title” should be **commented out**
  - “title” format should be **easy to spot visually**
- each **section** should perform a **single task**
  - separate data loading, from transformation, from modeling
- use as many **sections** as **individual tasks** are needed
- each task should maintain **execution independence** from other tasks
  - think **self-contained code** that should work even if the script is modified

(3) break down **tasks** in **sections**

```
# ======== LOAD DATA ========================================

confrontations = pd.read_excel(raw_data_confrontations,
    sheet_name=0, # first sheet (Python uses 0-based indexing)
    na_values="9999" # converting sentinel value to null
)

# ======= SOME DATA PROCESSING ==================================

forces_confrontations = format_table(forces_table_confrontations,
                                     forces_name_lookup)

forces_aggressions = format_table(forces_table_aggressions,
                                  forces_name_lookup)
```

(3) break down **tasks** in **sections**

ProTips:

- **if your script is getting too long** (too many sections):
  - it may be time to break it down into **multiple scripts,** each with a **differentiated purpose**
  - **execution independence** becomes even more important with multiple scripts (contain dependencies across scripts)

structured scripts in a nutshell!

- **define the purpose of your code** early on!
  - avoid doing the same task twice!
- group all **definitions** (of objects / functions) used throughout the script in a **common location**
- each script should perform **only one general task**
  - useful to **call additional scripts** from your script if/when needed
  - create a **global parameters** script if/when needed
  - if too many functions, create a **separate script defining all functions**

why we do this in Data Science?

1) removes **friction** – *teammates know and use same structure*
2) facilitates **debugging** your code – *you can easily find relevant code*
3) facilitates **testing** your code – *you can quickly check earlier steps*
4) facilitates **reusing** code in different projects – *(re)plug & play*
5) facilitates **handing off** code to teammates – *shared understanding*

don’t forget proper **syntax**

(1) write **informative** code

- are you this person?

```
# The "I'm in a hurry" starter pack
x = get_x()            # Thanks, that clears things up!
data2 = process(data) # Ah yes, data2, data's talented sibling
tmp = calculate_tmp() # "tmp" - the variable equivalent of "thingy"

# The "I got paid by the character" collection
def retrieve_and_return_all_user_records_from_the_postgresql_database():
    pass # I lost the will to live before finishing this

# The "I learned coding from a treasure map"
x1 = a + b
x2 = x1 * c
x3 = x2 / d
result = x3 # X marks the spot... but which spot?
```

(1) write **informative** code

- maybe you should be **this** person

```
# informative Python
user_count = len(users)
is_authenticated = check_token(token)
max_attempts = 3

def fetch_latest_orders(customer_id):
   return db.orders.filter(customer=customer_id).recent()

def calculate_shipping_cost(weight, distance):
    return weight * distance * 0.5

def get_user_email(user_id):
    return db.query(user_id).email
```

(1) write **informative** code

- use names that indicate **“direction”** to **avoid ambiguity**

```
# Bad - unclear or ambiguous
active = True
verified = check_user()
is_not_valid = check_input()
succeed = validate_all_conditions()

# Good - crystal clear states
is_admin = False
has_discount = cart.total > 100
can_edit = user.role in ["admin", "editor"]
will_expire_soon = expiry_date < (today + timedelta(days=7))
```

(1) write **informative** code

- start **function names** with **verbs**

```
# Bad - unclear or ambiguous
def report(data): # Generate? Print? Email? View?
    print(data)

def calculation(x, y): # "Do a calculation" - but WHICH?
    return x + y

# Good - clear actions
def calculate_total_price(items):
    return sum(item.price for item in items)

def send_email(recipient, message):
    smtp.send(recipient, message)
```

(2) write **readable** code

- improve the readability of your code with **spaces,** though never before a comma

```
# Bad - No breathing room
x=5+3*2
total=price+tax-discount
if user=="admin"and count>5:
    result=calculate(a,b,c)

# Good - Operators get personal space
x = 5 + 3 * 2
total = price + tax - discount
if user == "admin" and count > 5:
    result = calculate(a, b, c)
```

(2) write **readable** code

- **indent and align** your code to enhance readability

```
# Bad - Inconsistent indentation
def process_order(order):
  if order.is_valid:
      calculate_total(order)
  send_confirmation()    # What level is this?!
 log_order(order)        # Help!

# Good - Clear hierarchy
def check_user(user):
    if user:
        if user.is_active:
            if user.has_permission:
                return True
    return False
```

(2) write **readable** code

- be consistent in how you **format names**
  - **variables / functions –** `snake_case`
  - **constants / global objects –** `ALL_CAPS`

```
# Good - consistent and clean - joy to read!
user_name = "Alice"
user_age = 30
MAX_RETRIES = 5
DATABASE_HOST = "localhost"

def calculate_price(base_price):
    TAX_RATE = 0.08
    DISCOUNT_RATE = 0.1
    return base_price * (1 + TAX_RATE - DISCOUNT_RATE)
```

(3) write DRY code

- embrace your **lazy** side when writing code:
  - write **DRY** (**D**on't **R**epeat **Y**ourself) code
  - avoid **WET** (**W**rite **E**verything **T**wice) code

```
# WET (repetitive)
price1 = 10 * 1.08
price2 = 15 * 1.08
price3 = 20 * 1.08

# DRY (non-repetitive)
def add_tax(price):
    return price * 1.08
```

never forget **code syntax!!**

ProTips:

- **80 characters** should be the maximum length of any line in your code
- **never mix spaces and tabs** to indent your code
- **never use names of Python functions** for your own objects
- **good object / function name**: informative + easy to read + appropriate length
- consult **PEP 8** or **Google Python Style Guide** for **Pythonic style preferences**

always **document** your **code**

(1) with in-line comments

- in-line comments should **add knowledge** to your code, not repeat what the code does
- always **start comments** with **# followed by space**
- separate your code into distinguishable chunks using visually distinct characters like : or - or =
- include comments for any line of code **if meaning would be ambiguous** to someone other than yourself

(1) with in-line comments

```python
# ========== COFFEE ORDER PROCESSOR ==========

# --- Setup ---
order_total = 23.75
tip_threshold = 20.0

# --- Calculate tip ---
# GOOD: 20% tip for orders over $20 (keeps barista happy)
tip = order_total * 0.20

# --- Loyalty points ---
base_points = int(order_total * 10)
is_tuesday = True

# Tuesdays get 1.5x points per marketing's Q4 promotion
multiplier = 1.5 if is_tuesday else 1.0
total_points = int(base_points * multiplier)

# ========== Output ==========
print(f"Tip: ${tip:.2f} | Points: {total_points}")
```

(2) with `docstring` for functions

- **functions** require **additional detail** than regular code
- use `docstrings` – **long versions of comments** at the **start** of a function
- `docstrings` document:
  - **what the function should be doing**
  - **input parameters**
  - **outputs**
  - **examples**
- incredibly useful to **debug**, **maintain** or **update** functions

(2) with **docstring** for functions

```
def transform_enrich_data(base_event_data, state_names, forces_table, source_string):
    """
    Transform raw event data into analysis-ready format with location names and force identifiers.

    This function enriches sparse event records by:
    1. Adding human-readable state and municipality names from Census data (raw data only contains numeric codes - not
       useful for reports)
    2. Translating column names from Spanish to English (ensures accessibility for English-speaking analysts)
    3. Identifying which armed force was involved in each confrontation (critical for conflict analysis and attribution)
    4. Replacing missing values with 0 (simplifies downstream calculations and prevents NaN errors)

    Parameters
    ----------
    base_event_data : DataFrame
        Raw event database containing numeric location codes and Spanish columns
    state_names : DataFrame
        Lookup table mapping location codes to State/Municipality names
    forces_table : DataFrame
        Reference table identifying armed forces by event characteristics
    source_string : str
        Label identifying the origin/source of this data (e.g., 'ACLED_2024')

    Returns
    -------
    DataFrame
        Enriched event data with readable location names, English column headers, armed force identifiers added as new
        column, and all NaN values replaced with 0

    Examples
    --------
    >>> cleaned_data = transform_enrich_data(raw_events, census_names, force_lookup, 'UCDP')
    >>> cleaned_data.columns
    ['date', 'state_name', 'municipality', 'armed_force', 'casualties']
    """
```

ALWAYS document your code!!

ProTips:

- code shows **WHAT** you're doing, but comments should explain **WHY** you're doing it – *especially when it's not obvious*)
- if your code needs **too many comments**, you probably overdid it
  - you must simplify it when cleaning it up
- look into **Google Style Guide** for in-line comment and `docstring` inspiration

and don’t skip **code validation**

why do we need code validation?

- **important** to make sure that your code is **robust**:
  1) it's **doing** what it's **supposed to do**
  2) it's **not doing something else** (it shouldn't be doing)
  3) it does **not break** when **exceptions** occur
  4) it provides **informative messages** when errors occur
- **code validation** requires writing **tests** to ensure 1) to 4)
- **especially important** when your code will **integrate with a larger codebase**
  - no guarantee that it never will

(1) code does what it’s supposed to

- verify the **output** of your code is **what you intended**

```
# Bad - Looks correct but isn’t
def calculate_average(numbers):
    return sum(numbers) / 10 # Hardcoded 10 instead of len(numbers)!
result = calculate_average([1, 2, 3])
print(result)          # Returns 0.6 instead of 2.0 Nobody checked.

# Good - Verify expected output explicitly
def calculate_average(numbers):
    return sum(numbers) / len(numbers)

# Verify it does what it's supposed to assert
calculate_average([2, 4, 6]) == 4.0    # Basic case assert
calculate_average([1, 1, 1, 1]) == 1.0 # All same values assert
calculate_average([0, 10]) == 5.0      # Includes zero assert
calculate_average([-2, 0, 2]) == 0.0   # Negative numbers
```

(2) code does not go rogue

- verify the **output** of your code is <u>only</u> **what you intended**

```
# Bad - Does the job but modifies the original list!
def get_top_scores(scores, n):
    scores.sort(reverse=True) # Silently mutates the original!
    return scores[:n]

player_scores = [45, 92, 13, 78, 56]
top_3 = get_top_scores(player_scores, 3)
print(top_3)          # [92, 78, 56] - correct!
print(player_scores) # [92, 78, 56, 45, 13] - WAIT, this changed!

# Good - Does its job without touching the original
def get_top_scores(scores, n):
    return sorted(scores, reverse=True)[:n] # sorted() returns new list

player_scores = [45, 92, 13, 78, 56]
top_3 = get_top_scores(player_scores, 3)
print(top_3)          # [92, 78, 56] - correct!
print(player_scores) # [45, 92, 13, 78, 56] - untouched!
```

(3) code produces informative error messages

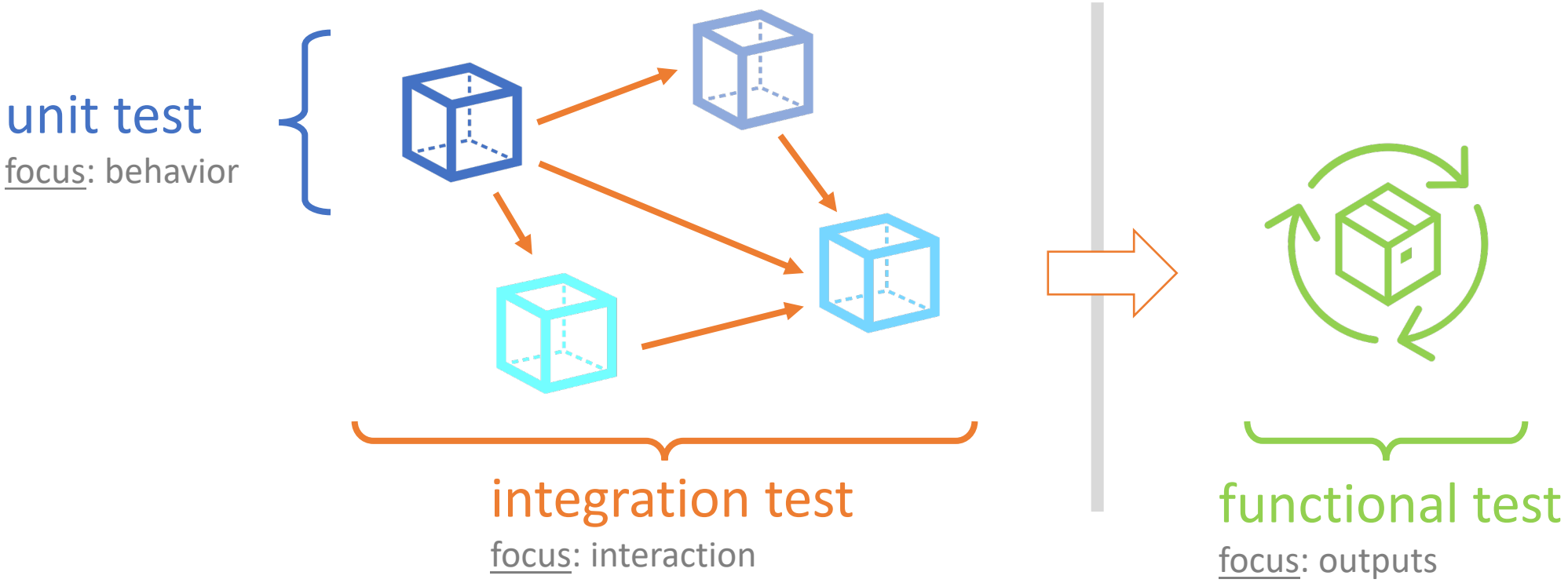
- verify your code **doesn't just fail** when it finds an **error**, but **handles the error** and gives **informative messages**

```
# Bad - Let errors bubble up
def divide_numbers(a, b):
    return a / b # ZeroDivisionError waiting to happen!

# Good – error tells you exactly what went wrong
def create_user(name, age):
    if not name:
        raise ValueError(
            "User name cannot be empty"
        )
    if age < 0:
        raise ValueError(
            f"User age cannot be negative. Received: {age}"
        )
```

(4) some pointers to **stress test** your code

```python
# When stress-testing any function, ask:

def my_function(param):
    """
    ✓ Does it handle None/null inputs?
    ✓ Does it handle empty collections ([], {}, "")?
    ✓ Does it handle negative numbers (if applicable)?
    ✓ Does it handle zero?
    ✓ Does it handle very large numbers?
    ✓ Does it handle wrong types (string instead of int)?
    ✓ Does it handle edge cases (first, last, only one)?
    ✓ Does it return consistent types?
    ✓ Do transformations produce expected results?
    ✓ Are errors handled gracefully?
    ✓ Do side effects actually occur?
    ✓ Can it be called multiple times safely?
    """
    pass
```

when your **code** is part of a **larger project**...



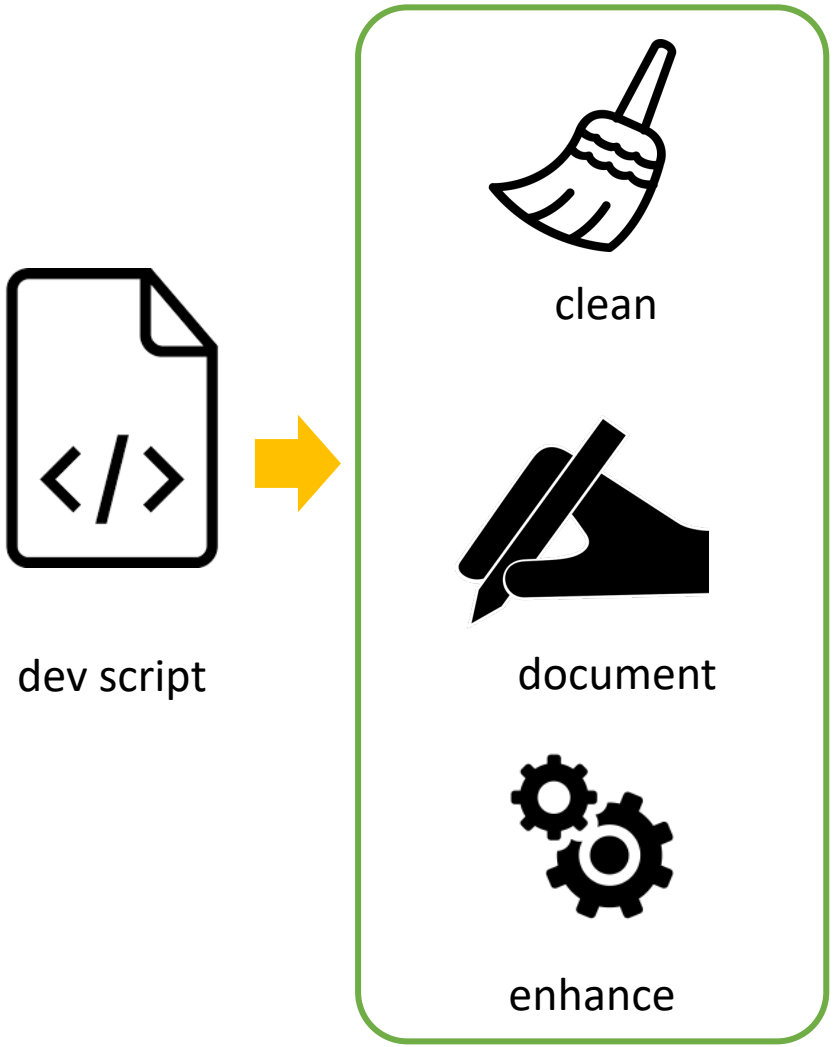
validate, validate, and **validate** your code!!

ProTips:

- if you find an error in your code, **correct it exactly where it happened**
  - NEVER fix it from a later chunk of code

workflow steps to live by

when done with your project – **step 1**



what do we mean by **cleaning** your code?

- produce code that will not **confuse** the next person to **read / use** your codebase
- **cleaning your code** usually involves:
  - removing **unused code** (*e.g.* commented out code)
  - removing **testing/validation code** (e.g. `print()` steps)
  - replacing **uninformative variable/function names** (*e.g.* `has_college_degree` instead of `education`)
  - **grouping code** in appropriate sections/modules (*e.g.* collect all library/package calls at same location)

clean

what do we mean by **documenting** your code?

- add **“knowledge”** to the information embedded in your code – do not describe what your code is doing
- **documenting your code** usually involves:
  - adding **informative in-line comments**
  - adding `doscstrings`
  - enhancing your `README.md` file
  - unpacking **project information: objective, structure, instructions, limitations**
  - identifying **bugs** that need to be fixed (especially to scale up)
  - detailing (potential) **future enhancements**

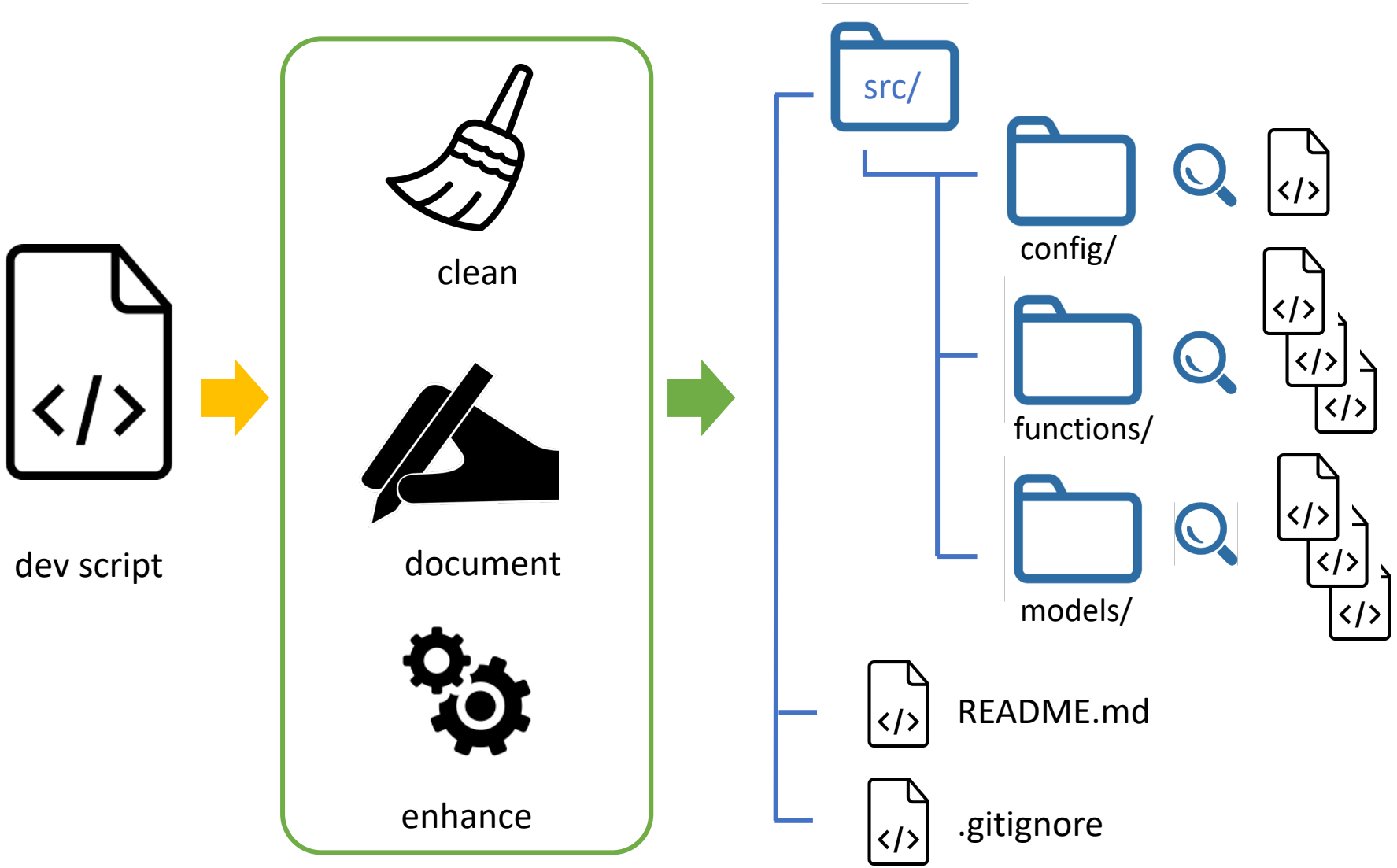
document

what do we mean by **enhancing** your code?

- **harden** your code to get it as close as possible to **production grade** code
- **enhancing your code** usually involves:
  - improving code **readability** (*e.g.* **PEP 8** standards in Python)
  - improving code **performance** (*e.g.* only use necessary memory resources)
  - reducing code **repetitiveness** (*e.g.* convert to generic functions)
  - applying **stress tests** and **edge cases** (*e.g.* try to "break" the code)
  - improving **error/exception handling** (*e.g.* use `try` or `except` in your functions)

enhance

when done with your project – **step 2**



what do we mean by **folder structure**?

- create / enhance a **folder structure** for your `src` code that makes sense for your project
  - `config/` – definitions, requirements
  - `functions/` – (collection of) function(s) script(s)
  - `models/` – for all your machine learning needs
  - `…/`
- (if needed) **decant** your **scripts** to fit your **folder structure**
- **test** and **validate!**

what goes on a `README.md` file?

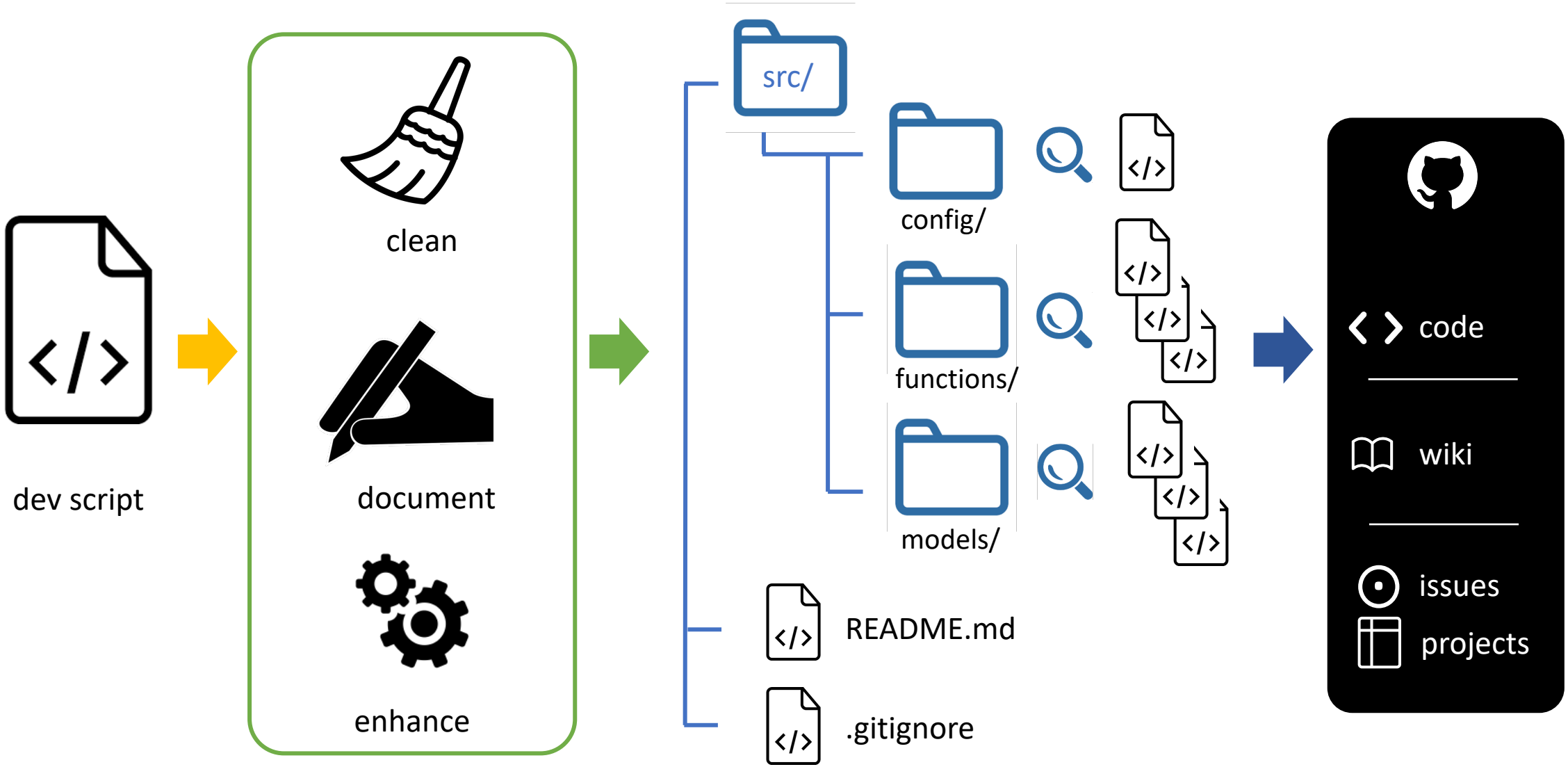
- the `README.md` file is the landing page for your GitHub repo
  - it's an **overview** of everything you need to know to use it
- a `README.md` file contains, at minimum:
  - **executive summary** of the project
  - project **structure** and how to **navigate** it
  - **limitations** of the project as it exists
  - where **additional documentation** lives (usually in GitHub wiki)
  - where to find known **bugs** and **enhancements** (usually in GitHub issues / project)

what goes on a `.gitignore` file?

- yes, every GitHub repo should include a .gitingore file!

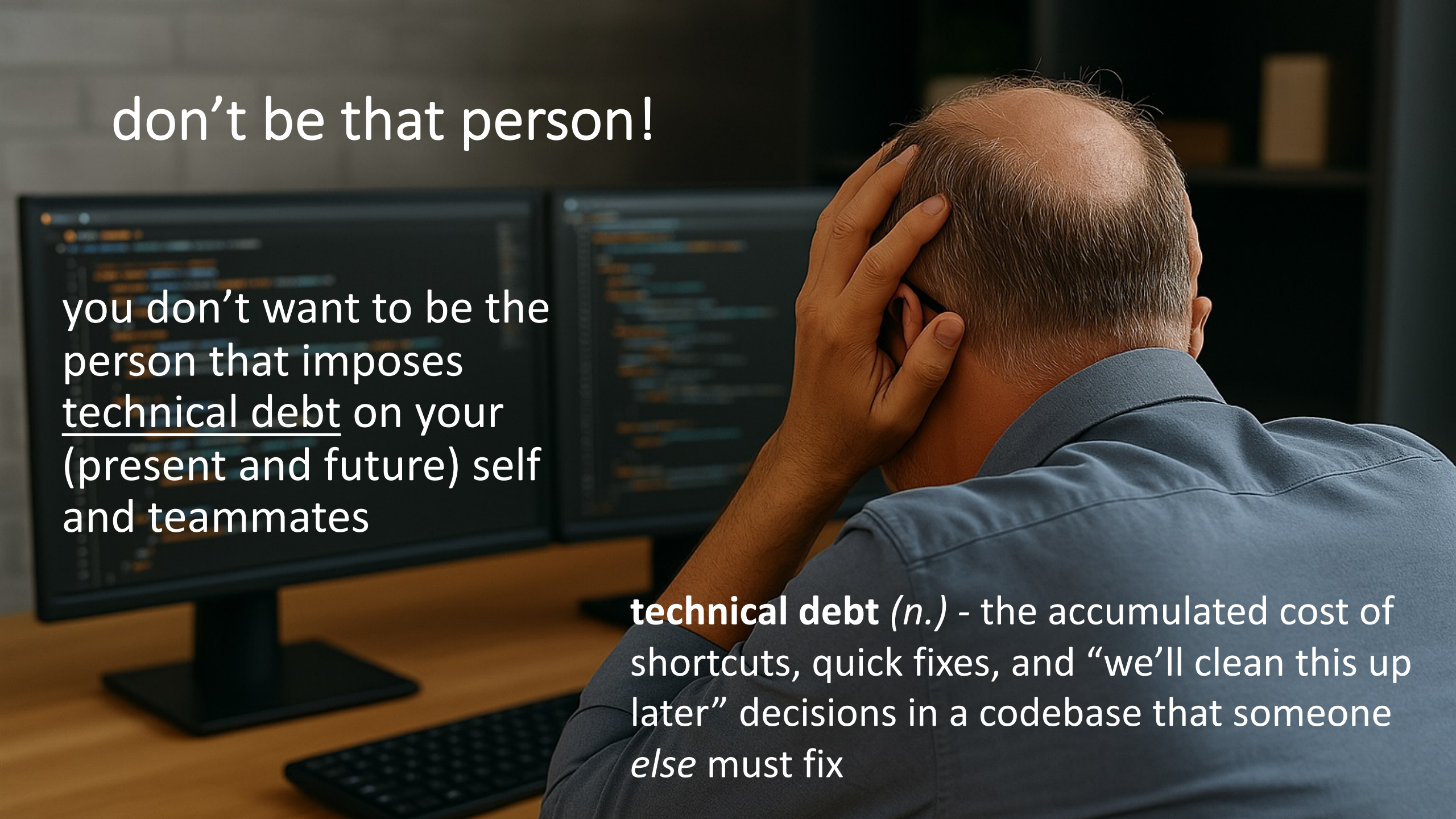
```
# Byte-compiled / optimized / DLL files
__pycache__/
*.py[codz]
*$py.class

# data sources
data/

# PyBuilder
.pybuilder/
target/

# Jupyter Notebook
.ipynb_checkpoints

# IPython
profile_default/
ipython_config.py
```

when done with your project – **step 3**



when done with your project – **step 3**

- send it off to live in GitHub!
  - `push` **cleaned**, **documented** and **enhanced** codebase to a GitHub (code) repo
  - add **documentation** to GitHub **wiki**
  - create **issues** – to detail **bug fixes** and/or **future enhancements** – in GitHub **issues**
  - if needed (and have time) group **issues** in GitHub **projects**
- minimal baseline for **handoff** and/or **scaling up**



don’t be that person!

you don’t want to be the person that imposes technical debt on your (present and future) self and teammates

technical debt *(n.)* - the accumulated cost of shortcuts, quick fixes, and “we’ll clean this up later” decisions in a codebase that someone *else* must fix

`commit` messages in git

`commit` with informative messages

- **remember**: `commit` **small chunks of logically grouped changes**
  - you may want to undo a change, but only that change
- message summarizes **what changed**
  - use imperative mood
    - [*this commit will*] `Rename income variable`
  - start with a **capital letter** and **do not end with a period**
  - maximum length: **50 characters**
- if you need to provide more detail on the **what** and **why**:
  - add a **paragraph** that wraps text at **72 characters**
- or add information in your **pull request**

in sum: basic principles to write "good" code

1. use **scripts for everything** you do
   - **NEVER** do things **manually**
2. organize your scripts in a sequence
   - separate **activities** in sections
   - keep an early section for **definitions**
   - call **other scripts** when necessary
3. write **efficient** (aka lazy) **code**
   - turn code used multiple times into **functions**
   - **re-use** functions: make them generic enough
   - always **validate** and **stress test** your code
4. rely on **version control** software (git)

can’t stress enough: follow workplace *idioms*

- make sure to understand all ***idioms*** at your workplace
  - **best practices**
  - **conventions**
  - **styles**
  - **ad hoc rules**
- **specifics** may be unique to your workplace, general principles apply universally

coding etiquette for (non-coder) Social Scientists

Marco Morales
marco.morales@columbia.edu

Nana Yaw Essuman
ne2388@columbia.edu

GR5069: Applied Data Science
for Social Scientists

Spring 2026
Columbia University